CELEBAL INTERNSHIP

NAME – ANMOL BHATIA

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Traditional MapReduce has a few limitations that make it less suitable for today's big data workloads. One of the biggest drawbacks is that it writes intermediate data to disk after every processing stage, which slows down execution because disk operations are much slower than memory. It is also not efficient for iterative tasks like machine learning, where the same data needs to be processed multiple times. In addition, writing MapReduce programs is more complex since developers need to define separate Map and Reduce functions.

Apache Spark addresses these issues by processing data in memory whenever possible, which greatly improves performance. It also provides simple DataFrame and SQL APIs, making data processing easier to write and understand. Because of its speed, ease of use, and support for analytics, machine learning, and streaming, Spark has become a preferred choice for modern big data processing.

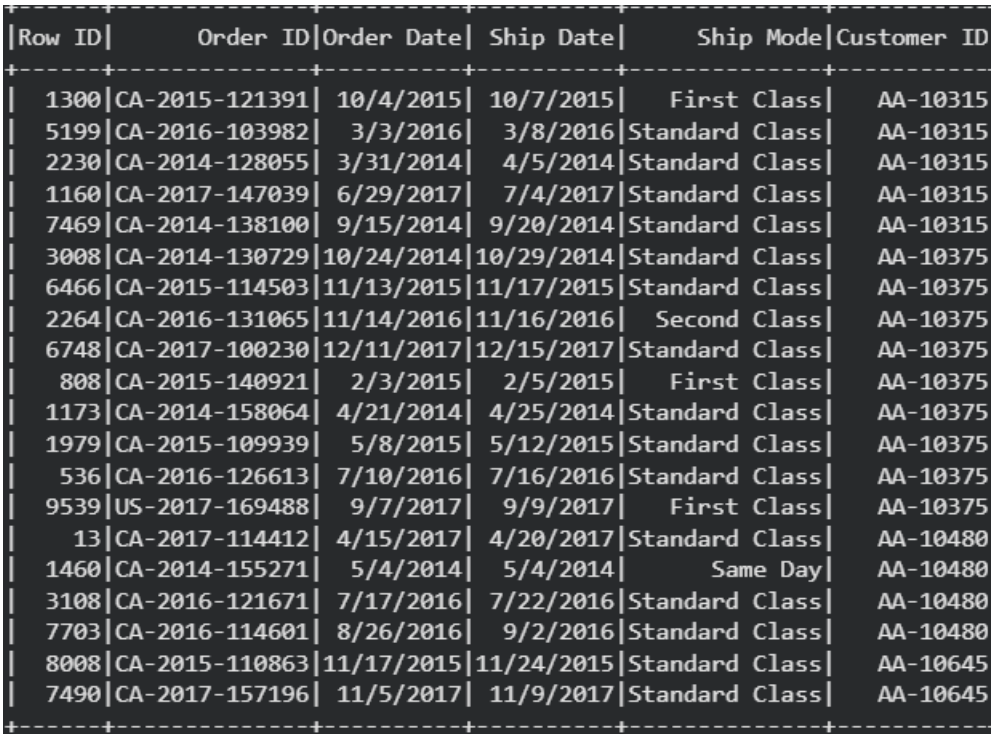
Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Spark speeds up iterative machine learning tasks by keeping data in memory instead of reading it from disk after every iteration. In systems like MapReduce, the same data is loaded from disk repeatedly, which increases execution time. Spark loads the data once and reuses it from memory, reducing disk I/O and improving performance. It also supports caching, allowing frequently used datasets to stay in memory for faster processing.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

df_clean = df.dropDuplicates(["Customer ID", "Order Date"])

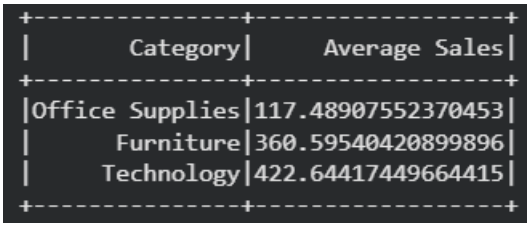
df_clean.show()



Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID
1300	CA-2015-121391	10/4/2015	10/7/2015	First Class	AA-10315
5199	CA-2016-103982	3/3/2016	3/8/2016	Standard Class	AA-10315
2230	CA-2014-128055	3/31/2014	4/5/2014	Standard Class	AA-10315
1160	CA-2017-147039	6/29/2017	7/4/2017	Standard Class	AA-10315
7469	CA-2014-138100	9/15/2014	9/20/2014	Standard Class	AA-10315
3008	CA-2014-130729	10/24/2014	10/29/2014	Standard Class	AA-10375
6466	CA-2015-114503	11/13/2015	11/17/2015	Standard Class	AA-10375
2264	CA-2016-131065	11/14/2016	11/16/2016	Second Class	AA-10375
6748	CA-2017-100230	12/11/2017	12/15/2017	Standard Class	AA-10375
808	CA-2015-140921	2/3/2015	2/5/2015	First Class	AA-10375
1173	CA-2014-158064	4/21/2014	4/25/2014	Standard Class	AA-10375
1979	CA-2015-109939	5/8/2015	5/12/2015	Standard Class	AA-10375
536	CA-2016-126613	7/10/2016	7/16/2016	Standard Class	AA-10375
9539	US-2017-169488	9/7/2017	9/9/2017	First Class	AA-10375
13	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480
1460	CA-2014-155271	5/4/2014	5/4/2014	Same Day	AA-10480
3108	CA-2016-121671	7/17/2016	7/22/2016	Standard Class	AA-10480
7703	CA-2016-114601	8/26/2016	9/2/2016	Standard Class	AA-10480
8008	CA-2015-110863	11/17/2015	11/24/2015	Standard Class	AA-10645
7490	CA-2017-157196	11/5/2017	11/9/2017	Standard Class	AA-10645

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
from pyspark.sql.functions import expr, avg

df_fixed = df.withColumn( "Sales",expr("try_cast(Sales as double)"))

q4_result = (df_fixed.filter(df_fixed["Region"] == "West").groupBy("Category")

    .agg(avg("Sales").alias("Average Sales"))

)

q4_result.show()
```



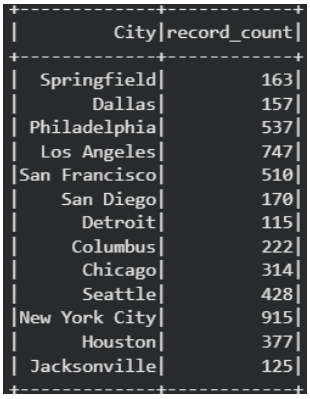
Category	Average Sales
Office Supplies	117.48907552370453
Furniture	360.59540420899896
Technology	422.64417449664415

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

.na.drop() removes rows that contain null values, while .na.fill() replaces null values with a specified value instead of deleting the rows. Using .na.fill() helps preserve the data by filling missing values where appropriate.

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
from pyspark.sql.functions import count, col

q6_result = ( df.groupBy("City").agg(count("*").alias("record_count"))

    .filter(col("record_count") > 100)

)

q6_result.show()
```



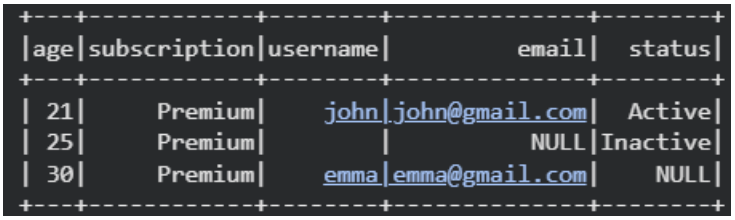
City	record_count
Springfield	163
Dallas	157
Philadelphia	537
Los Angeles	747
San Francisco	510
San Diego	170
Detroit	115
Columbus	222
Chicago	314
Seattle	428
New York City	915
Houston	377
Jacksonville	125

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are immutable, which means they cannot be changed after they are created. When performing data cleaning tasks such as dropping or renaming columns, Spark creates a new DataFrame instead of modifying the existing one. This helps maintain data consistency and allows transformations to be applied safely without affecting the original data.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
from pyspark.sql.functions import col

q8_result = sample_df.filter((col("age").between(18, 30)) &

    (col("subscription") == "Premium")

)

q8_result.show()
```



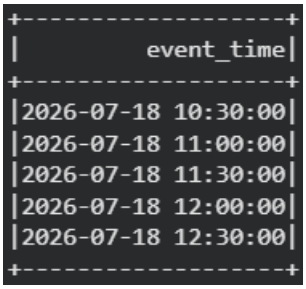
age	subscription	username	email	status
21	Premium	john	john@gmail.com	Active
25	Premium		NULL	Inactive
30	Premium	emma	emma@gmail.com	NULL

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Null values should usually be handled before using functions like sum() or avg() because they can affect the accuracy of the result. Spark may ignore null values during aggregation, which can make the final value misleading if many records are missing. Cleaning or filling nulls first makes the calculation more consistent and helps ensure that the result represents the dataset properly.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
from pyspark.sql.functions import col

from pyspark.sql.types import TimestampType

q10_result = sample_df.withColumn("event_time",
col("raw_timestamp").cast(TimestampType())

).drop("raw_timestamp")

q10_result.select("event_time").show()
```



Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

A shuffle is the process where Spark redistributes data across different partitions during operations like groupBy(), join(), or distinct(). This is needed because related data must be brought together before the operation can be completed. Shuffle is considered a wide transformation because it involves moving data between partitions, which increases network communication and makes the operation slower than narrow transformations.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
from pyspark.sql.functions import col

q12_result = sample_df.filter(col("email").isNotNull() &

    (col("username") != ""))

q12_result.show()
```

age	subscription	username	email	status	raw_timestamp	price
21	Premium	john	john@gmail.com	Active	2026-07-18 10:30:00	100.0
17	Basic	alex	alex@gmail.com	NULL	2026-07-18 11:00:00	NULL
30	Premium	emma	emma@gmail.com	NULL	2026-07-18 12:00:00	500.0
35	Basic	mike	mike@gmail.com	Active	2026-07-18 12:30:00	150.0

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

```
from pyspark.sql.functions import min, max, avg

q13_result = df_fixed.agg(
    min("Sales").alias("Minimum Sales"),
    max("Sales").alias("Maximum Sales"),
    avg("Sales").alias("Average Sales")
)

q13_result.show()
```

Minimum Sales	Maximum Sales	Average Sales
0.444	22638.48	234.41818199917006

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Using inferSchema=True can be risky when the dataset contains inconsistent date formats. Spark decides a column's data type by examining the values, so mixed formats may cause the column to be detected as a string instead of a date or timestamp. Some values may also become null when they are later converted. It is safer to define the schema manually and clean or standardize the date values before casting them.

Q15: Write a final processing pipeline that:

1. **Filters out duplicates.**
2. **Fills null prices with 0.**
3. **Groups by store_id to calculate total revenue.**

```
from pyspark.sql.functions import col, sum

q15_result = (df_fixed.dropDuplicates().na.fill({"Sales": 0})

    .groupBy("Customer ID")

    .agg(sum("Sales").alias("Total Revenue")))

q15_result.show()
```

```
+-----------+------------------+
|Customer ID|     Total Revenue|
+-----------+------------------+
|   VW-21775|          6112.702|
|   RR-19315| 615.9320000000001|
|   PB-19210|           132.738|
|   EM-13960|           933.704|
|   MY-17380|2242.6119999999996|
|   MS-17530|           475.656|
|   KH-16630|3918.9660000000003|
|   AH-10690| 7888.294000000001|
|   BD-11500|          4408.297|
|   SW-20275|1966.6499999999999|
|   JF-15490|1082.9180000000001|
|   JF-15415|          2371.448|
|   PH-18790|           729.648|
|   IM-15070|          4930.474|
|   PW-19240|          3878.315|
|   KM-16225|          2134.248|
|   JH-15985|          7954.998|
|   OT-18730|          3818.624|
|   KD-16615|           2243.51|
|   NW-18400| 7234.013999999999|
+-----------+------------------+
```